

Algoritmos y Estructuras de Datos.

- Clase 3 - Módulos y Objetos -

Módulos en Python

Para que nos sirven? Nos permiten definir nuestras propias funciones, bloques de código, métodos, objetos, tipos de datos, etc. Cuando desarrollamos, aunque es posible hacerlo de manera individual, **requiere mucho tiempo y esfuerzo**, además de se debe considerar la (posible) **falta de eficiencia u optimización** del código.

El comando **import** nos permite utilizar funciones y objetos definidos fuera de las funciones pre-definidas en Python.

Por qué estas funciones no estas dentro de Python? Son muchas y muy específicas a cada dominio. A raíz de esto, los siguientes problemas aparecen:

- Uso ineficiente de memoria.
- Conflictos de nombres.

Usar *módulos* nos permite trabajar de forma colaborativa, en equipos. De esta manera se podrán abordar proyectos de gran magnitud.

Cómo usar **import** ?

Hay dos formas de usar el comando **import**.

Importar solo un módulo.

```
import nombre_módulo
```

Luego podremos llamar a la cualquier función `func` definida dentro del módulo `nombre_módulo`, de la siguiente forma: `nombre_módulo.func`.

In []:

```
#Ejemplo
import random
random.randrange(100)
```

In []:

```
import numpy
numpy.pi #numpy.pi es la aproximación del número pi.
```

De forma técnica, el comando **import** `nombre_módulo`, `nombre_módulo` es una nueva variable del tipo *módulo*. Luego, las funciones y/o métodos definidas en el módulo pueden ser llamados o invocados desde nuestro código.

In []:

```
print(type(random)) # Tipo : module (módulo)
print(type(random.randrange)) # Tipo : method (método)
print(type(numpy.pi))
```

In []:

```
randrange(100) # Esto genera un error del tipo "NameError" porque la función ran
drange no esta definida.
```

También es posible cambiar el nombre de los módulos, por otro, generalmente más corto.

```
import nombre_módulo as nuevo_nombre
```

In []:

```
import math as m # el nommbre 'math' no es asociado al modulo (el nombre de vari
able esta "libre")
                # m es el objeto que contiene las funciones del módulo math.
print(m.pi) # funciona...
math.pi # NameError
```

Importar objetos desde un módulo

La otra manera de importar funciones (objetos, métodos, etc.) definidas en un módulo es asignando un nuevo monbre para ellas:

```
from nombre_módulo import algo_1, algo_2, ...
as nuevo_mombre_1, nuevo_nombre_2, ...
```

In []:

```
from math import pi as sliceofpie # sliceofpie es un número con el valor de mat
h.pi
from math import pi # el comando "as" es opcional
pi==sliceofpie # Son Iguales
```

Si queremos importar un módulo de manera "completa", es decir todas las funciones, objetos, etc, invocamos el siguiente comando:

```
from nombre_módulo import *
```

In []:

```
#import math
#math.pi

from random import *
randrange(100)

from math import *
pi
```

In []:

```
import random
for x in dir(random): #Funciones provistas por Python (built-in functions).
    print (x, end= " - ")

help(random)
```

Aunque los módulos mas usados, toman en cuenta la definición de nombres de manera cuidadosa para no generar conflictos. Es una buena practica evitar importar "todo" (utilizando asterisco) si importaremos mas de un módulo.

Algunos de los módulos mas utilizados

Nota: algunos de estos módulos vienen provistos dependiendo de que versión y via que método instalamos Python. Los módulos qué no esten presentes, deben ser instalados manualmente o mediante la herramienta "pip"

Módulo math

Para la mayoría de los módulos "comunes" de Python, la documentación es extensiva y clara. Ver Documentación del módulo math: <https://docs.python.org/es/3.6/library/math.html> (<https://docs.python.org/es/3.6/library/math.html>)

La libreria math contiene funciones comunes (logaritmos, exponenciales, trigonometricas, ...) y algunas constantes (pi, e, ...), además de algunas otras definiciones, por ejemplo `inf` ($+\infty$) y `nan` "un número de tipo flotante" que "No Representa a un Número" (Not A Number).

In []:

```
from math import inf, nan

print(inf+inf)
print(1/inf)
print(inf-inf) # "Indeterminado" aunque no genera error
print()
print(nan)
```

In []:

```
1.0/0.0 # Infinito no es un resultado valido.
```

In []:

```
from math import log
log(0) #no -inf definido (solo +inf)
```

Ejercicio : Calcular la media Aritmetico-Geometrica. Formula:

Sea $u_0 = 1$, $v_0 = x$. Si u_n y v_n estan definidas, definimos el siguiente termino como:
$$\begin{cases} u_{n+1} = \frac{u_n + v_n}{2} \\ v_{n+1} = \sqrt{u_n v_n} \end{cases}$$

Podemos probar que ambas secuencias convergiran en el punto fijo $f(x)$. Calcular una aproximación de la función $f(x)$ para $x \in [0, 0.01, \dots, 0.99, 1]$.

In []:

```
#Ejercicio Resuleto:
counter=0
import math
def AGM(x):
    global counter
    #c=0
    u,v=1,x #Initialization
    while u!=(u+v)/2:
        counter+=1
        u=(u+v)/2
        v=math.sqrt(u*v)
    return u

print(AGM(5))

counter
```

In []:

```
for x in range(0,101):
    print(AGM(x/100), end=",")
```

In []:

```
counter
```

Módulo numpy

Su Documentación completa está disponible aqui: <http://www.numpy.org> (<http://www.numpy.org>).

Particularmente interesante para problemas mathematicos. Nos ofrece un nuevo tipo de dato "arreglo" (array), tambien llamados "vectores", cuando todos los elementos son del mismo tipo.

In []:

```
import numpy as np
```

In []:

```
v=np.array([[1,1],[1,1]])# Crea un arreglo de 2 x 2 (es decir, una matriz).
                        # v= [ 1 | 1
                        #      1 | 1 ]
print(v)
print([[1,1],[1,1]])
type(v)
```

In []:

```
v+=v
print(v)
```

In []:

```
v=np.array([[1,2],[3,4]])
print(v)
print(v[0][1]) # Podemos indexar los elementos de un array
v[0][1]=5
print(v)
w=v           # Los arrays son objetos "mutables".
w[0][1]=3
print(w, '\n', v)
```

In []:

```
np.array([[True,2.0],[0+1j,0.1],[3+7*1j,-np.pi]]) # La *función* array normaliza
ra los datos                                     # que ingresamos por parametro
s.
```

In []:

```
np.zeros([2,3,4,3]) # podemos crear arreglos multi-dimensionales
```

Además tenemos provisto el tipo *Matríz*.

In []:

```
M=np.matrix([[0,1j],[1j,0]]) # Tipo : matrix
print(M.conjugate()) # Retorna w1 conjugado de la matriz.
print(M*M)
```

In []:

```
N=M           # Las matrices son elementos mutables.
N[0,1]=2713
print(M)
```

Existen muchos más métodos relacionados al manejo de matrices y álgebras, contenidos en el *sub-módulo* `linalg`.

In []:

```
from numpy import linalg as LA
eigvalues,eigvectors=LA.eigh(np.matrix([[10,1],[1,2]])) #eigh calcula los eigenvalues de matrices simetricas-hemitaneas
print(eigvalues)
```

Módulo scipy

Su Documentación completa está disponible aqui: <https://www.scipy.org> (<https://www.scipy.org>). Contiene a los módulos numpy (análisis numerico), sympy (computación simbolica), y otras herramientas útiles que veremos luego.

Módulo random

Como lo indica su nombre, se utiliza para generar valores (u objetos) aleatorios.

In []:

```
import random
```

In []:

```
random.random() # Retorna un número de punto flotante aleatorio entre 0 y 1 (igual al que la calculadora)
```

In []:

```
random.randrange(0,4) # Retorna un entero aleatorio entre 0 y 3 (incluido)
```

In []:

```
Meses=['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec']
random.shuffle(Meses) #Mezcla los meses (la lista Meses ha sido cambiada)
Meses
```

In []:

```
random.sample(Meses,k=12) # Retorna una muestra de 5 elementos de la población Meses
#Comparar con las opciones: random.shuffle(L) and random.sample(L,len(L))
```

In []:

```
#Ejercicio : Crear una matriz simetrica de tamaño 100 y calcular sus eigenvalues
from numpy import matrix
import random
ListtoMatrix=[]
for x in range(0,100):
    Ligne=[]
    for y in range(0,x):
        Ligne.append(ListtoMatrix[y][x])
    for y in range(x,100):
        Ligne.append(random.randrange(-10,10))
    ListtoMatrix.append(Ligne)
eigval,eigvect=LA.eigh(matrix(ListtoMatrix))
len(eigval)
eigval
```

In []:

```
#Ejercicio: Paradoja del Cumpleaños. Dentro de un grupo de 10 personas, asumiend
o que
#sus cumpleaños siguen una distribución normal sobre los 365 días de año. Calcula
r una aproximación
#de la probabilidad que dos personas cumplan años el mismo día.
#Mismo ejercicio, pero para 30 personas.

N=10000 #Numero de iteraciones (tests)
Npeople=10 #Número de personas.
S=0 #+1 si no hay cumpleaños que iguales, 0 en otro caso.
for ntest in range(0,N):
    Lbirthday=[]
    for x in range(0,Npeople):
        birthday=random.randrange(0,366)
        if birthday in Lbirthday: # Verificar si dos personas tienen el mism
o cumpleaños
            test=0 # Mantendremos S = 0.
            break # Salimos del ciclo.
        else:
            Lbirthday.append(birthday) #Añadir el cumpleaños a la lista de cumpl
eaños para ese día.

    if len(Lbirthday)==Npeople: # Si la lista de cumpleaños Lbirthday es igual
a Npeople, entonces todas
# las personas tienen diferentes cumpleaños.
        test=1

    S+=test # Añadimos los resultados del test a la variable S

print(S/N) # Mostrar porcentaje.
```

In []:

```
# Ejercicio : Supongamos que hacemos una "tirada" de 10 dados.
# Ganaremos $200 si la suma de los valores esta entre los 10 y 23.
# Perderemos $1600 si la suma de los valores es 24 o 48.
# Ganaremos $2500 si la suma de los valores es 42.
# Perderemos $500 en cualquier otro caso.
# Deberiamos jugar a este juego?

N=10000 #Número de iteraciones o "tests"
S=0     #Suma total del dinero ganado/perdido

for ntest in range(0,N):

    dicesvalue=0          #Inicialización

    for ndice in range(0,10):      #" tiramos los dados 10 veces"
        dicesvalue+=random.randrange(1,7) # "tirada" de **un** dado

    if dicesvalue<= 23:    # Aplicamos las reglas del juego
        S+=200
    elif dicesvalue==10 or dicesvalue==23:
        S-=1600
    elif dicesvalue==42:
        S+=2500
    else:
        dicesvalue-=500

print(S/N) # El promedio S/N es una buen estimador de la "espranza" de las ganancias.
```

Importar módulos escritos por el usuario

Podemos importar nuestras propias "Clases" de módulos.

Clases en Python

En Python, una *Clase* nos permite definir nuestros propios tipos de objetos. Debemos pensar en una Clase, cómo un "template" de nuestros objetos, donde definiremos sus **atributos** (variables internas) y/o **métodos** (funciones internas). Ambas pueden ser invocadas regularmente utilizando `obj.name`.

Definición

In []:

```
class MyfirstClass(): pass
    # Atributos
    # Métodos

x=MyfirstClass() # Define un objeto del Tipo 'MyfirstClass'
type(x)
```

In []:

```
xy=MyfirstClass()  
xy.name='Pierre' # Luego de realizada esta asignación "xy.name" es el "name" del  
objeto 'xy'  
print(xy.name)      # Notar que el atributo 'name' no estaba definido anteri  
ormente en MyfirstClass.  
z=xy  
z.name='Paul' # los atributos son objetos mutables.  
print(xy.name)
```

De manera similar a las funciones, podremos definir valores por defecto, para la instanciación de un nuevo objeto:

In []:

```
class Ratio():  
    "Número Racional"  
    def __init__(self,numerator,denominator): # Siempre usar "self" para referir  
se a los atributos                                     # del objeto (variables internas)  
        self.num=numerator  
        self.den=denominator
```

In []:

```
q=Ratio(0,1) # Crea una variable "q" de tipo "Ratio" , donde el numerador es 0 y  
el denominador es 1  
print(q.num)  
print(q.den)
```

Métodos asociados al objeto

Dentro de la Clase definimos los métodos (funciones internas a los objetos) que los objetos requieren para realizar computaciones, acciones, etc.

In []:

```
class Ratio():  
    "rational number"  
    def __init__(self,numerator,denominator):  
        self.num=numerator  
        self.den=denominator  
    def val(self):  
        return self.num/self.den
```

In []:

```
val(Ratio(0,1)) # Esto no funciona, recordar que 'val' es un método y no una fun  
ción.
```

In []:

```
q=Ratio(0,1)  
print(q.val())  
print(Ratio.val(q))
```

Sobrecarga de operadores/comandos (Overloading)

La "sobrecarga" de una función, operador, método, ya definido nos permite cambiar su significado. Por ejemplo, nos gustaría re-definir el operador "suma" "+", para nuestra nueva clase de objetos.

In []:

```
class Ratio():
    "rational number"
    def __init__(self,numerator=0,denominator=1):
        self.num=numerator
        self.den=denominator

    # Methods
    def val(self):
        return self.num/self.den

    #Overloading operators
    def __add__(self,other):    # Sobrecarga del operador '+'.
        return Ratio(self.num*other.den+self.den*other.num,self.den*other.den)
    def suma(self,other):      # Sobrecarga del operador '+'.
        return Ratio(self.num*other.den+self.den*other.num,self.den*other.den)
    def __str__(self):         # Sobrecarga necesaria para la función 'print'.
        return str(self.num)+'/'+str(self.den)  #Cómo devolvemos una cadena de c
aracteres, para que
                                                #la función print muestre algo p
ara nuestro objeto.
    def __mul__(self,other):    # Sobrecarga del operador '*'.
        return Ratio(self.num*other.num,self.den*other.den)
    def __int__(self): #Sobrecarga de la función 'int'.
        return int(self.val())

#print(obj)----> print(str(obj))
```

In []:

```
q1=Ratio(2,3)
q2=Ratio(1,3)
q=q1 + q2 # q3= q1.suma(q2) # q3 = Ratio.suma(q1,q2)
print(q.num,q.den) # No es muy bonito, podemos arreglarlo?
print(q)
```